

18-01-2026

Agenda: Python - II

- String properties •
- operators - II •
- control flow •
- control flow - II (advance) •
- non primitive data type. •

String properties

"Data science" → `< class 'str' >` ← function
↓ properties
↓ methods

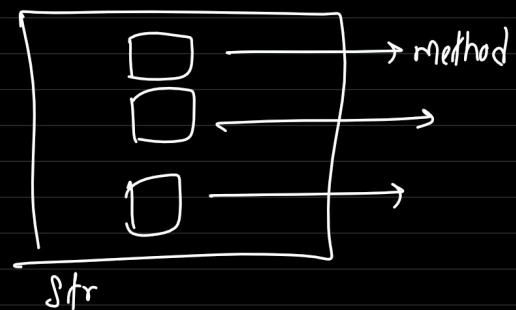
class
↑
House → doors
→ window
→ room
→ floor

glass() →

str → strip()
→ lstrip()
→rstrip()

print() →
id() →
type() →
int() float() str() } function

→ function (type())



car → class → Drive
→ open door

House → class → ~~Drive~~
→ live
→ room

door (↓) → function

methods → property of something is called a method

function → does some job.

datatype
string → strip()
→ lstrip()
→ rstrip()

int → ~~strip()~~

→ we use . to call its property.

function → not related to datatype

→ does something

→ cannot be used with " "

To remove white space from string we use below methods:

→ lstrip() → removes space from left side

→ rstrip() → removes space from right side

→ strip() → lstrip() + rstrip()

name = "Data"

name = name.lstrip().rstrip()

"Data" .rstrip() → "Data"

name = (name).lstrip().rstrip()

print(9) → 12

print(12) →

Developer → google term → 'Data', 'data', 'daTa', 'data'

Age
→ '27', '28', '40', '30' → int('27') → 27
'27' →

int('27'.strip())

Operators

Arithmetic Operator

Relational operator / comparison operator

Relational operators: → relation of two values & we use operator to check if it is true or not.
True False

$12 > 3$ greater than

$12 == 12$ equal

$3 < 12$ smaller than

greater than or equal to :

$12 \geq 3$

smaller than or equal to :

$3 \leq 12$

not equal to :

$3 \neq 12$

$12 > 3 \rightarrow \text{True}$

$12 < 3 \rightarrow \text{False}$

$12 \geq 3 \rightarrow \text{True}$

$12 \leq 3 \rightarrow \text{False}$

$12 \neq 12 \rightarrow$
↓
opposite of $==$

$12 == 12 \rightarrow \text{True}$
↓
False

$3 == 3 \rightarrow \text{True}$

$3 \geq 3 \rightarrow \text{True}$

"Data" > "Data"

"Data" == "Data"

Assignment operators :

$$a = 2$$

$$q \vdash 2 \longrightarrow a = a + 2$$

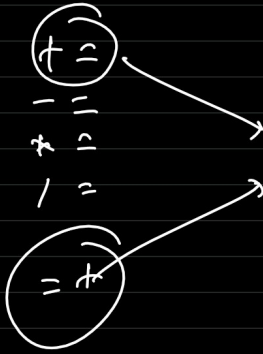
$$\begin{array}{c} \downarrow \downarrow \\ q-2 \Rightarrow q \sim q-2 \end{array}$$

$$a \neq 2 \Rightarrow a = a/2$$

$$a * 2 \Rightarrow a = a * 2$$

109M

Mona)



Logical operator

Truth Table:

operator	A	B	output
And	0	0	0
	1	1	1
	0	1	0
	1	0	0

and $\rightarrow$ True, when both are True
else False (0).

operator	A	B	output
OR	0	0	0
	1	1	1
	0	1	1
	1	0	1

OR $\rightarrow$ if any one is true $\rightarrow$ True
else false

operator	A		output
NOT	0		1
	1		0

NOT $\rightarrow$
True $\rightarrow$ false
False $\rightarrow$ True

operator	A	B	output
XOR	0	0	0
	1	1	0
	0	1	1
	1	0	1

XOR $\rightarrow$ only true when only
one of them is true

Bitwise operators : -

&, |, ^

$\begin{matrix} 2^2 & 2^1 & 2^0 \\ 4 & 2 & 1 \end{matrix} \rightarrow 7 (1-7)$

1 $\rightarrow$	0	0	1 $\rightarrow$
2 $\rightarrow$	0	1	0
3 $\rightarrow$	0	1	1 $\rightarrow$
4 $\rightarrow$	1	0	0
7 $\rightarrow$	1	1	1

$\begin{array}{r} 011 \quad 3 \\ 001 \quad 1 \\ \hline 001 \rightarrow 1 \end{array}$

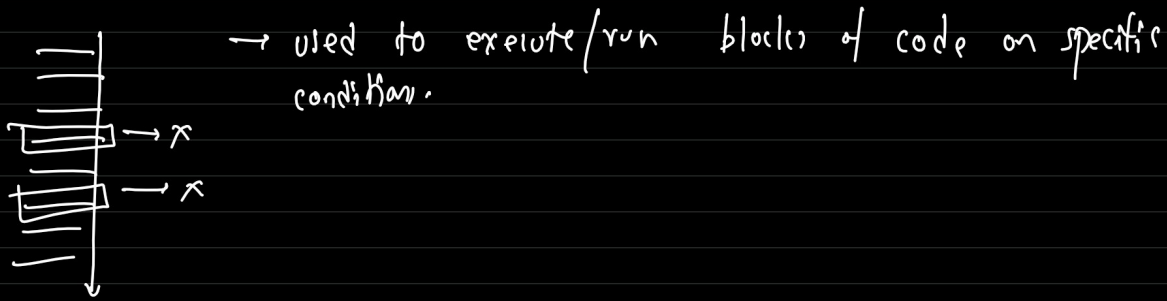
$\begin{array}{r} 011 \quad 3 \\ 001 \quad 1 \\ \hline 011 \rightarrow 3 \end{array}$

$\begin{array}{r} 011 \quad 3 \\ 100 \quad 4 \\ \hline 111 \end{array}$
 $\downarrow$
 NOT

$\wedge \rightarrow$ XOR

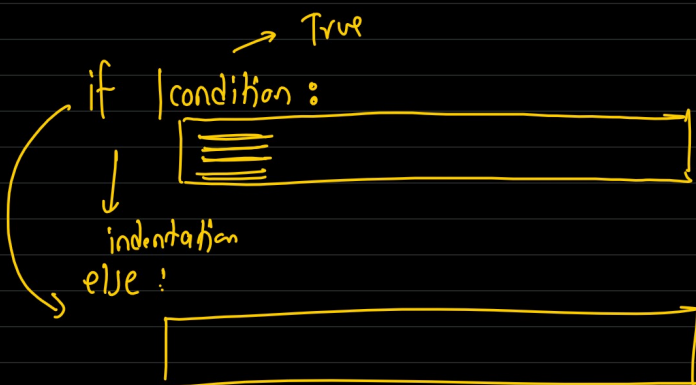
Control Flow : —

Control the flow of the program



Task 1: apple: 4

if → whether 4 apples or more are present in the store
we will buy apple
then →
we will not buy apple



if condition:
[]
else:
[]



if condition:
[]



~~else:~~
~~[]~~

1 Tab = 4 space

Task: application/tool $\longrightarrow$ connect to a wifi

①

```
if wifi_avail:
    connect
```

②

```
if wifi_avail:
    connect
else:
    tell user, no wifi available
```

Control flow - II

```
if cond:
    do something
else:
    do something
```

```
if cond:
    do something
```

```
if cond:
    do something
elif cond:
    do something
...
elif cond:
    do something
else:
    do something
```

```
elif cond:
    do something
```

if cond: $\longrightarrow$ starts control flow block

if cond: $\longrightarrow$ start control flow block

if cond: $\longrightarrow$ start

elif cond: $\longrightarrow$ end

if condition $\longrightarrow$ start

primary data types of python → primitive data type.

→ int, float, str, bool

non-primitive data types:

→ list, dict, tuple, set

list = [1, 2, 3, 4, 5] → values are comma separated
variable name square bracket closing square bracket

array → [1, 2, 3, 4, 5] → int
(not python) ["Hello", "world"] → str

list → [1, 2, 3, 3.0, 4.0, "Mona"]

people-register:
↓
building entry
register

name:	①
phone:	②
address:	③
in:	④
out:	⑤
reason:	⑥
flat:	⑦

} → 1 person

person-1 → [①, ②, ③, ④, ⑤, ⑥, ⑦]

register = []

register.append (person-1)

↓ ↓
[1, 2, 3, 4]
[1, 4, 5, 8]
[]
[[1, 2, 3, 4]] [[1, 2, 3, 4], [1, 4, 5, 8]]
↑ ↑ ↑